

Backend Documentation

Full technical reference for explaining the API — architecture, auth, modules, security, data, deploy.

Stack: **Node.js ≥20** · **Express 5** · **TypeScript** · **MongoDB (Mongoose)** · **JWT** · **Nodemailer** · **Google Auth** · **Gemini (optional)**

Live API: <https://ssc-0p96.onrender.com> · Generated for walkthrough / interview / handover

1. What this backend is

ExamPrep API powers a competitive-exam prep product (SSC, Banking, Railways, UPSC, CAT, State PSC). It serves authentication, syllabus/notes, daily drills, timed topic tests, full mocks, analytics data, competition (battle) mode, reminders, exam config, and an optional AI explainer.

- **Repo path:** `backend/`
- **Entry:** `server.ts` → `createApp()` in `src/app.ts` → `/api` router
- **Dev:** `npm run dev` (`tsx watch`) · **Prod:** `npm run build` → `node dist/server.js`

2. Table of contents

1. Boot & request pipeline
2. Auth system (register, OTP, Google, JWT)
3. API modules & endpoints
4. MongoDB models
5. Security
6. Environment variables
7. Scripts & data
8. Deploy (Render)
9. How to explain it in 2 minutes

3. Boot & request pipeline

3.1 Process start (`server.ts`)

1. Load `.env` only when not on Render (hosted uses dashboard env).
2. `validateEnv()` — production requires Mongo, `JWT_SECRET` (≥32), `FRONTEND_URL`.
3. `connectDB()` — hosted exits if Mongo fails; local can run degraded.
4. Mount Express app, start reminder cron (every minute unless `REMINDER_CRON=0`).
5. Listen on `PORT` (default 5000).

3.2 Middleware order (`app.ts`)

```
Helmet → Permissions-Policy → Compression
→ CORS gate → CORS
→ JSON / urlencoded (2mb)
→ mongoSanitize → HPP → Morgan
→ Rate limit (hosted only, 600 / 15 min on /api)
→ /api router
→ / , /health , /keepalive
→ 404 → error handler
```

Path	Purpose
GET /	API identity
GET /health	Readiness — 503 if Mongo down
GET /keepalive	Liveness / free-tier wake ping — always 200
GET /api/health	Same readiness under /api

3.3 API zones

- **Public:** `/api/auth/*` (with own limiters), `/api/auth/google-config`
- **Protected:** everything else under `/api` after global `requireDb` + `requireAuth`

4. Auth system

This is the most important story to tell in an interview: secure session design with OTP, Google, and tokenVersion invalidation.

4.1 Flows

```
Register → email OTP (pending user in OtpChallenge) → verify OTP creates User → password login → JWT
Login (unverified) → force OTP again → then password login
Google → verify ID token (preferred) or auth-code → upsert user → JWT
Forgot password → OTP → reset (+ bump tokenVersion) → login
Logout → tokenVersion++ → old JWTs rejected
```

4.2 Design choices worth explaining

- **Register does not create User immediately** — password hash + profile sit in OTP `pendingData` until email verify.
- **OTP codes are hashed** (HMAC with pepper), TTL 10 minutes, max 5 attempts.
- **Forgot password never reveals** whether an email exists (anti-enumeration).
- **Gmail aliases** (dots / plus / googlemail) matched so OTP/login don't break.
- **Admin role only from `ADMIN_EMAIL`** — never trusted from the client.
- **JWT carries `tv` (tokenVersion)** — logout & password reset invalidate all sessions.
- **Google preferred path = ID token** (no client secret). Code flow optional if `GOOGLE_CLIENT_SECRET` set.
- **Passwords:** bcrypt cost 12; legacy SHA-256 upgraded on successful login.

4.3 Auth endpoints

Method	Path	Notes
GET	/api/auth/google-config	Public clientId + codeFlowEnabled
POST	/api/auth/register	Start registration + send OTP
POST	/api/auth/login	Email/username + password
POST	/api/auth/otp/request	Resend verify OTP
POST	/api/auth/otp/verify	Verify email (no JWT yet)
POST	/api/auth/password/forgot	Send reset OTP
POST	/api/auth/password/reset	New password + invalidate sessions
POST	/api/auth/google	credential or code → session
GET	/api/auth/me	Current user + progress
POST	/api/auth/logout	Bump tokenVersion
POST	/api/auth/progress	Save topic test attempt
POST	/api/auth/mock-progress	Save mock attempt
GET	/api/auth/progress/export	CSV (self or admin all)
GET	/api/auth/mock-progress/export	CSV (self or admin all)
GET	/api/auth/admin/summary	Admin institute stats

5. Feature modules & endpoints

5.1 Study — /api/study

Subjects, topics, notes, topic tests, vocabulary bank.

Method	Path	Who
GET/POST	/subjects	Any user
DELETE	/subjects/:subjectName	Any user
GET/POST	/subjects/:subjectName/topics	Any user
GET	/topics/:topicId/notes	Any user
GET	/topics/:topicId/test	Any user
PUT/DELETE	/topics/:topicId	Any user
GET	/vocab	Any user
POST / PUT	/vocab, /vocab/bulk, /vocab/:id	Admin

5.2 Drill — /api/drill

Method	Path	Purpose
GET	/next	Next weighted practice item (TCS / vocab / tables)
POST	/verify	Check answer; store DrillPerformance
GET	/related	Related questions for a topic

5.3 Mock — /api/mock

Method	Path	Who
GET /	List mocks	Any user
POST /	Create mock	Admin
GET /:id	Full paper	Any user
DELETE /:id	Delete	Admin

5.4 Competition — /api/competition

Method	Path	Purpose
GET	/questions	Random TCS sample for battle
POST	/submit	Save score (clamped); return PB + rank
GET	/leaderboard	Top 10 by subject

5.5 AI — /api/ai

POST /explain — wrong-answer explanation (Gemini → Pollinations fallback → static template). Rate limit 30 / 15 min.

5.6 Exam config — /api/exam-config

GET / list configs; PUT /:examId admin upsert subject lists per exam (SSC, Banking, ...).

5.7 Questions — /api/questions

Method	Path	Who
GET	/tcs/stats	Admin
POST	/tcs/bulk	Admin bulk upload
POST	/add	Any user contribute to bank

5.8 Reminders — /api/reminders

CRUD reminders; list/mark/test notifications. Cron emails + creates in-app AppNotification .

5.9 Prep — /api/prep

Status ping + legacy sticky notes (admin write). Primary notes live on study topics.

6. MongoDB models (collections)

Model	Used for
User	Accounts, role, googleId, tokenVersion
OtpChallenge	Hashed OTP + pending register data (TTL)
Progress	Topic test attempts (red/yellow/green)
MockProgress	Full mock attempts
Subject / Topic / Question	Syllabus tree + topic MCQs
Vocab	Word power / idioms / OWS for drills
TCSQuestion	Large MCQ bank (drills + battle)
DrillPerformance	Adaptive weighting (30-day TTL)
MockTest	Full papers with sections
CompetitionScore	Battle leaderboard
ExamConfig	Per-exam subject mapping
Reminder / AppNotification	Schedules + inbox

Static seed/reference JSON also exists under `backend/data/chs1/` (topics + drills).

7. Security checklist

- Helmet + COOP for Google popups + FedCM Permissions-Policy
- Strict CORS allowlist (prod Vercel + FRONTEND_URL / FRONTEND_URLS)
- Global + auth + OTP + AI rate limits on hosted
- Body size cap 2mb; HPP; Mongo operator / prototype pollution sanitize
- express-validator on auth/progress bodies
- No stack traces to clients in production
- CSV export escapes formula injection (=, +, -, @)
- `trust proxy` on Render for correct client IPs

8. Environment variables

Required in production

Variable	Why
MONGODB_URI	Database
JWT_SECRET	≥32 chars signing key
FRONTEND_URL	CORS + email links

Important optional

Variable	Why
ADMIN_EMAIL	Only admin promotion path
GOOGLE_CLIENT_ID / SECRET	Google sign-in
SMTP_*	OTP + reminder email
GEMINI_API_KEY	AI explanations
OTP_PEPPER	OTP hash pepper
FRONTEND_URLS	Extra CORS origins
REMINDER_CRON=0	Disable cron
KEEPALIVE_URL	Render free-tier ping target

9. Scripts

Command	Purpose
npm run seed	Seed subjects / topics / questions
npm run seed-vocab-options	Vocab distractors + SSC idioms/OWS
npm run migrate-data	Legacy embedded topics → collections
npm run clean-db	Dedupe TCS questions
npm run clean-topic-dupes	Dedupe topic questions

10. Deploy (Render)

- Blueprint: `render.yaml` — web service `ssc-backend`, free Node, health check `/health`
- Build: `npm install && npm run build` (postinstall also builds)
- Start: `node dist/server.js`
- Cron every 5 minutes hits `/keepalive` to reduce free-tier sleep
- Frontend on Vercel proxies `/api/*` → this Render host

11. Folder map

```
backend/
├── server.ts                # process entry
├── render.yaml
├── src/
│   ├── app.ts              # Express factory
│   ├── routes/index.ts     # /api map
│   ├── config/             # db + env
│   ├── shared/middleware/  # auth, db, validate, sanitize, errors
│   ├── modules/
│   │   ├── auth/
│   │   ├── study/
│   │   ├── drill/
│   │   ├── mock/
│   │   ├── competition/
│   │   ├── ai/
│   │   ├── exam-config/
│   │   ├── questions/
│   │   ├── reminders/
│   │   └── prep/
│   └── types/
├── scripts/                # seed / migrate / cleanup
└── data/                  # CHSL JSON reference content
```

12. How to explain this backend (2-minute pitch)

1. "I built a production-style Express + Mongo API for an exam-prep product."
2. "Auth is layered: email OTP registration, Google ID-token login, JWT with tokenVersion invalidation, rate limits, and hashed OTPs."
3. "Domain modules cover syllabus, adaptive drills, mocks, battle mode, reminders, and admin exam config."
4. "Security is intentional: Helmet, CORS allowlist, sanitize, validation, admin only via ADMIN_EMAIL."
5. "Deployed on Render with health + keepalive; frontend on Vercel talks through an /api rewrite."